Práctica 2. Código Seguro: del dato al fallo

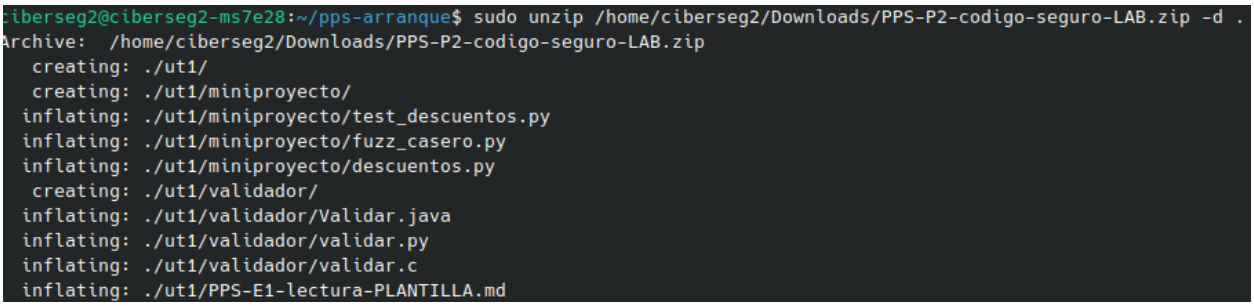
Nombre y apellidos: David Simonyan Demchuk

Hasta ahora hemos estado con Git y Docker. A partir de hoy los usamos como herramientas de trabajo y nos metemos en lo que de verdad importa en este módulo: leer código buscando por dónde puede entrar un ataque, y demostrar con pruebas que un fallo está arreglado y no va a volver. La práctica es individual.

Antes de empezar, un aviso: validar.py ejecuta órdenes de verdad en tu ordenador. Si pruebas cosas raras, que sean inofensivas (id, whoami). Nada de borrar ficheros.

1. **Prepara el entorno**
   Descarga PPS-P2-codigo-seguro-LAB.zip del Drive (carpeta 01 ALUMNADO) y descomprímelo dentro de tu carpeta pps-arranque. Después crea un entorno virtual e instala pytest y coverage:

   cd pps-arranque
   unzip ~/Descargas/PPS-P2-codigo-seguro-LAB.zip -d .



   python3 -m venv .venv .
   .venv/bin/activate
   pip install pytest coverage

```
ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ python3 -m venv .venv
ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ . .venv/bin/activate
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ pip install pytest coverage
Collecting pytest
  Downloading pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting coverage
  Downloading coverage-7.17.0-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata
(8.1 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Downloading iniconfig-2.3.1-py3-none-any.whl.metadata (2.8 kB)
Collecting packaging>=22 (from pytest)
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Downloading pluggy-1.7.0-py3-none-any.whl.metadata (5.0 kB)
Collecting pygments>=2.7.2 (from pytest)
  Downloading pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
Downloading pytest-9.1.1-py3-none-any.whl (386 kB)
Downloading pluggy-1.7.0-py3-none-any.whl (27 kB)
Downloading coverage-7.17.0-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (261 kB)
Downloading iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Downloading packaging-26.3-py3-none-any.whl (129 kB)
Downloading pygments-2.21.0-py3-none-any.whl (1.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.3/1.3 MB 38.0 kB/s eta 0:00:00
```

Después de crear la carpeta virtual instalamos el putet coverage.

Asegúrate de que la carpeta .venv no se va a subir al repositorio:

grep -qx '.venv/' .gitignore || echo '.venv/' >> .gitignore

```
.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ sudo nano .gitignore
sudo: authenticate] Password:
.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ grep -qx '.venv/' .gitignore || echo '.venv/' >> .gitignore
.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque$
```

Después de crear el .gitignore metemos el .venv/ al archivo para que no lo suba al repositorio.

Si algo falla en este paso, avísame en cuanto te pase y sigue con el apartado 2.

2. Del dato de entrada al sink (PPS-E1)

Entra en la carpeta del validador y ejecuta estas dos órdenes:

cd ut1/validador

python3 validar.py ana

python3 validar.py "' OR '1'='1"

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ cd ut1/validador
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/validador$ python3 validar.py ana
True
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/validador$ python3 validar.py "' OR '1'='1"
Traceback (most recent call last):
  File "/home/ciberseg2/pps-arranque/ut1/validador/validar.py", line 35, in <module>
    print(buscar_usuario(sys.argv[1] if len(sys.argv) > 1 else "ana"))
          ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ciberseg2/pps-arranque/ut1/validador/validar.py", line 30, in buscar_usuario
    return eval(regla)
  File "<string>", line 1
    len('' OR '1'='1') > 0
           ^^
SyntaxError: invalid syntax. Is this intended to be part of the string?
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/validador$
```

Entramos en la carpeta de ut1/validar y al hacer validar.py ana nos da true

Fíjate en qué pasa con la segunda y en qué línea se para el programa. ¿Qué crees que significa?

Lo que ocurre es que en la línea 30, sale un error en return eval. El problema ocurre porque la entrada del usuario ejecuta una expresión mediante eval(). Y esto permite la ejecución de código no deseado o errores de sintaxis.

Ahora lee los tres ficheros (validar.py, Validar.java y validar.c). En cada uno tienes que encontrar por dónde entra el dato (la entrada), dónde puede hacer daño (el sink), si hay alguna validación y si sirve para algo, y por qué falla teniendo en cuenta cómo se ejecuta ese lenguaje: interpretado, máquina virtual o compilado.

Copia la plantilla y rellénala. Al final pon también cómo lo arreglarías:

cd ..

cp PPS-E1-lectura-PLANTILLA.md PPS-E1-lectura.md

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1$ sudo cp PPS-E1-lectura-PLANTILLA.md  PPS-E1-lectura.md
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1$ sudo nano PPS-E1-lectura.md
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1$
```

Para rellenar el documento que acabamos de copiar, primero tenemos que ver los 3 validar.

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/validador$ ls
Validar.java  validar.c  validar.py
```

(FALTA POR HACER)

3. Los tests pasan, pero… (PPS-E2)

Entra en el miniproyecto y pasa los tests midiendo la cobertura:

cd miniproyecto

coverage run -m pytest -q

Entramos en la carpeta miniproyectos y ejecuto el test para ver que nos indica, nos pone que 3 passed y 1 warning.

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/validador$ cd ../miniproyecto
```

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$ coverage run -m pytest -q
...                                                                      [100%]
3 passed in 0.00s
```

Los 3 tests marcan que están passed

coverage report -m

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$ coverage report -m
Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
descuentos.py           6      1    83%   24
test_descuentos.py     12      0   100%
-------------------------------------------------
TOTAL                  18      1    94%
```

Lee la incidencia #17 que hay al principio de descuentos.py. Apunta la cobertura que te sale. ¿Hay algún test que se dé cuenta del fallo?

El archivo descuentos.py tenía cobertura del 83%. Tenía 6 instrucciones y solo ha ejecutado 5 y 1 no se ejecutó por un error. Por lo que la cobertura total del proyecto es del 94%.

No, los 3 tests que hay han pasado sin ningún fallo, pero no hay incidencia 17. Los tests actuales no comprueban los descuentos no válidos. Por eso hace falta añadir una prueba que se dé cuenta de esos fallos.

4. Test de regresión (PPS-E2)

Al final de test_descuentos.py escribe una función test_regresion_incidencia_17 que compruebe que un descuento del 150 % y otro del -5 % lanzan ValueError (usa pytest.raises). Pasa los tests. El tuyo tiene que fallar. Si te sale bien a la primera, es que no está comprobando el fallo. Cuando lo veas fallar, arregla aplicar_descuento en descuentos.py y vuelve a pasarlos hasta que estén todos bien.

Primero tenemos que abrir el test_descuentos.py y añadir el test de regresión.

```
# --- TU TEST DE REGRESIÓN (incidencia #17) va aquí debajo.
def test_regresion_incidencia_17():
        with pytest.raises(ValueError):
                aplicar_descuento(100, 150)

        with pytest.raises(ValueError):
                aplicar_descuento(100, -5)
```

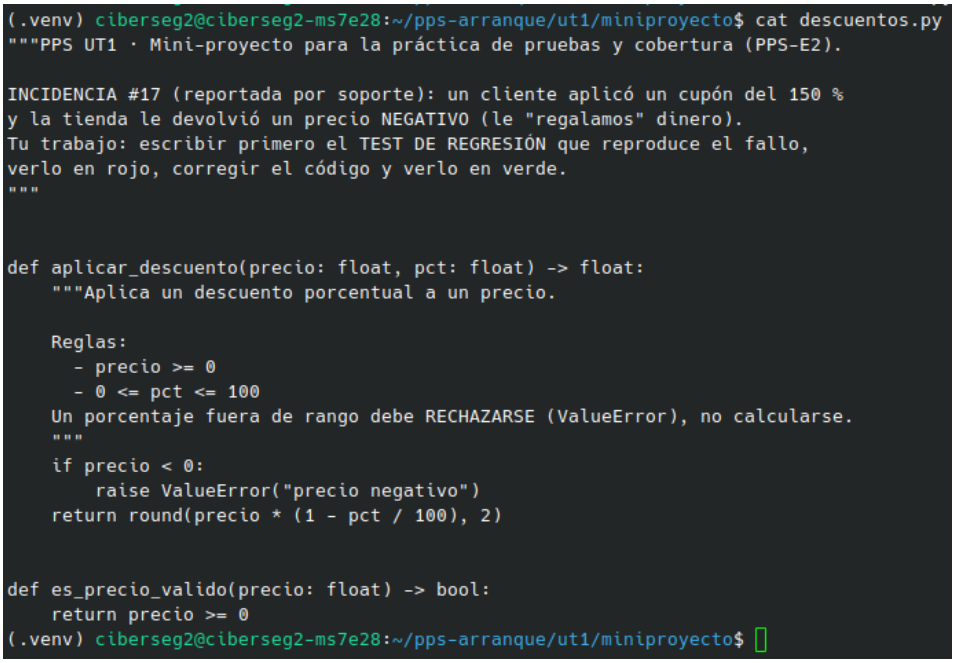
Al final del archivo en el apartado que nos indica ponemos el test de regresión. Y ahora ejecutamos los tests.

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$ coverage run -m pytest -q
...F                                                                                                                [100%]
================================================= FAILURES =================================================
_____________________________________ test_regresion_incidencia_17 _____________________________________

    def test_regresion_incidencia_17():
>       with pytest.raises(ValueError):
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^
E    Failed: DID NOT RAISE ValueError

test_descuentos.py:25: Failed
=========================================== short test summary info ============================================
FAILED test_descuentos.py::test_regresion_incidencia_17 - Failed: DID NOT RAISE ValueError
1 failed, 3 passed in 0.02s
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$
```

Y efectivamente nos da error que es lo que tendría que ocurrir.



```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$ cat descuentos.py
"""PPS UT1 · Mini-proyecto para la práctica de pruebas y cobertura (PPS-E2).

INCIDENCIA #17 (reportada por soporte): un cliente aplicó un cupón del 150 %
y la tienda le devolvió un precio NEGATIVO (le "regalamos" dinero).
Tu trabajo: escribir primero el TEST DE REGRESIÓN que reproduce el fallo,
verlo en rojo, corregir el código y verlo en verde.
"""


def aplicar_descuento(precio: float, pct: float) -> float:
    """Aplica un descuento porcentual a un precio.

    Reglas:
      - precio >= 0
      - 0 <= pct <= 100
    Un porcentaje fuera de rango debe RECHAZARSE (ValueError), no calcularse.
    """
    if precio < 0:
        raise ValueError("precio negativo")
    return round(precio * (1 - pct / 100), 2)


def es_precio_valido(precio: float) -> bool:
    return precio >= 0
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$
```

Y si miramos el descuentos.py vemos la indicencia #17. Y lo que tenemos que hacer es cambiar la parte defectuosa por la buena.

```python
"""PPS UT1 · Mini-proyecto para la práctica de pruebas y cobertura (PPS-E2).

INCIDENCIA #17 (reportada por soporte): un cliente aplicó un cupón del 150 %
y la tienda le devolvió un precio NEGATIVO (le "regalamos" dinero).
Tu trabajo: escribir primero el TEST DE REGRESIÓN que reproduce el fallo,
verlo en rojo, corregir el código y verlo en verde.
"""


def aplicar_descuento(precio: float, pct: float) -> float:
    """Aplica un descuento porcentual a un precio.

    Reglas:
      - precio >= 0
      - 0 <= pct <= 100
    Un porcentaje fuera de rango debe RECHAZARSE (ValueError), no calcularse.
    """
    if precio < 0:
        raise ValueError("precio negativo")
    if not 0 <= pct <= 100:
        raise ValueError("descuento fuera de rango")
    return round(precio * (1 - pct / 100), 2)


def es_precio_valido(precio: float) -> bool:
    """Comprueba si un precio es válido."""
    return precio >= 0
```

Después de cambiar eso volvemos a hacer la prueba para ver si todos los tests pasan.

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$ coverage run -m pytest -q
....                                                                                [100%]
4 passed in 0.00s
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$
```

5. Fuzzing

Con la función ya arreglada, ejecuta:

python3 fuzz_casero.py

Primero ejecutamos el fuzzing.

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$ python3 fuzz_casero.py
RESULTADO RARO nan: precio=nan pct=100
RESULTADO RARO nan: precio=nan pct=100
RESULTADO RARO nan: precio=nan pct=0
RESULTADO RARO nan: precio=nan pct=1e-09
RESULTADO RARO nan: precio=nan pct=1e-09
RESULTADO RARO nan: precio=nan pct=-0.0
RESULTADO RARO nan: precio=nan pct=0
RESULTADO RARO nan: precio=nan pct=-0.0
RESULTADO RARO nan: precio=nan pct=68.88881690722656
RESULTADO RARO nan: precio=nan pct=1e-09

10 fallos en 20000 intentos
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$
```

Entramos en el archivo de fuzz_casero.py con nano y vemos.

Este programa prueba 20.000 combinaciones de valores raros contra tu función y avisa cuando algo no cuadra. Mira qué encuentra. Después:

1.Completa la función resultado_valido() (donde pone TODO). Piensa qué tiene que cumplir siempre un precio después de aplicarle un descuento. Te puede venir bien el módulo math.

```python
def resultado_valido(precio, resultado):
    return (
        math.isfinite(precio)
        and precio >= 0
        and math.isfinite(resultado)
        and 0 <= resultado <= precio
    )
```

El resultado_valido ya está solucionado.
2. Arregla aplicar_descuento para que rechace esos valores.

Ahora editamos el archivo descuentos.py y arreglamos los descuentos para rehacer esos valores.

```python
def aplicar_descuento(precio: float, pct: float) -> float:
    if not math.isfinite(precio):
        raise ValueError("el precio debe ser finito")

    if not math.isfinite(pct):
        raise ValueError("el descuento debe ser finito")

    if precio < 0:
        raise ValueError("precio negativo")

    if not 0 <= pct <= 100:
        raise ValueError("descuento fuera de rango")

    resultado = round(precio * (1 - pct / 100), 2)

    if not math.isfinite(resultado) or resultado < 0:
        raise ValueError("el resultado no es válido")

    return resultado


def es_precio_valido(precio: float) -> bool:
    return math.isfinite(precio) and precio >= 0
```

3. Añade un test de regresión para lo que has encontrado.
Has terminado cuando el fuzzing dé 0 fallos y todos los tests pasen.

```python
def test_regresion_valores_no_finitos():
    import pytest

    valores_no_finitos = [
        float("nan"),
        float("inf"),
        float("-inf"),
    ]

    for valor in valores_no_finitos:
        with pytest.raises(ValueError):
            aplicar_descuento(valor, 10)

        with pytest.raises(ValueError):
            aplicar_descuento(100, valor)
```

Volvemos a ejecutar los tests y todos los tests deberian de dar por pasados.

```
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$ coverage run -m pytest -q
....                                                                     [100%]
4 passed in 0.00s
(.venv) ciberseg2@ciberseg2-ms7e28:~/pps-arranque/ut1/miniproyecto$
```

Si te atascas

Si te sale externally-managed-environment, estás usando el pip del sistema. Activa el entorno virtual con . .venv/bin/activate.

Si te sale No module named pytest, seguramente has abierto otro terminal y no has activado el entorno virtual.

Si te sale ModuleNotFoundError: No module named 'descuentos', no estás dentro de ut1/miniproyecto.

6. Entrega

Contesta las preguntas de abajo y sube la carpeta ut1 a tu repositorio. Antes de hacer el commit, comprueba con git status que no aparece .venv.

git add ut1

git commit -m "Práctica 2: PPS-E1 y PPS-E2"

git push

```
ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ git add ut1
ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ git commit -m "Práctica 2: PPS-E1 y PPS-E2 completadas"
[master (root-commit) f1c0360] Práctica 2: PPS-E1 y PPS-E2 completadas
 11 files changed, 263 insertions(+)
 create mode 100644 ut1/PPS-E1-lectura-PLANTILLA.md
 create mode 100644 ut1/PPS-E1-lectura.md
 create mode 100644 ut1/miniproyecto/.coverage
 create mode 100644 ut1/miniproyecto/__pycache__/descuentos.cpython-314.pyc
 create mode 100644 ut1/miniproyecto/__pycache__/test_descuentos.cpython-314-pytest-9.1.1.pyc
 create mode 100644 ut1/miniproyecto/descuentos.py
 create mode 100644 ut1/miniproyecto/fuzz_casero.py
 create mode 100644 ut1/miniproyecto/test_descuentos.py
 create mode 100644 ut1/validador/Validar.java
 create mode 100644 ut1/validador/validar.c
 create mode 100644 ut1/validador/validar.py
```

```
ciberseg2@ciberseg2-ms7e28:~/pps-arranque$ git push --set-upstream origin master
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 16 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (16/16), 8.96 KiB | 8.96 MiB/s, done.
Total 16 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:      https://github.com/DavidSimonyan533/Pps-arranque/pull/new/master
remote:
To https://github.com/DavidSimonyan533/Pps-arranque.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
ciberseg2@ciberseg2-ms7e28:~/pps-arranque$
```

Preguntas

2. **¿Cuál es el sink más peligroso de validar.py? ¿Por qué?**

Eval () porque cualquier texto que ponga el usuario lo puede ejecutar directamente como código python. Esto permite que un atacante ejecute código malicioso.

3. **¿Qué cobertura tenía descuentos.py con el fallo dentro?**

Tenía un 83%, faltaba una línea por ejecutar porque los test antiguos no comprobaban los errores.

4. **¿Qué mensaje te dio pytest cuando tu test fallaba?**

Failed: DID NOT RAISE <class 'ValueError'>. Significa que el programa devolvió un resultado incorrecto en lugar de lanzar el error de tipo ValueError que exigía el test.

5. **¿Qué valores encontró el fuzzing que tus tests no habían probado?**

Números decimales extremos, porcentajes negativos y porcentajes superiores al 100% combinados con precios flotantes, lo que provocaba fallos de precisión o precios negativos.

6. **URL de tu repositorio.**

https://github.com/DavidSimonyan533/Practica-2.Codigo-seguro